

Click & Align Subtitle Tool

Developer Documentation

Extension ID:	com.planetread.clickandalign.subtitle
Bundle Version:	1.0.0
Host Application:	Adobe Premiere Pro 25.0+
CEP Runtime:	CSXS 12.0
Document Date:	2025

Table of Contents

1. Overview & Purpose	3
2. Architecture	3
3. Layer Reference	4
4. UI → ExtendScript Map	4
5. State Model	5
6. Caption Export Flow	5
7. Function Reference — main.js	6
8. Function Reference — subtitles.jsx (Utilities & I/O)	7
9. Function Reference — subtitles.jsx (Marking & Export)	8
10. Mark Start State Machine	9
11. Mark End Behaviour	9
12. Extension Manifest	10
13. Debugging Guide	10
14. Known Issues & Follow-ups	11
15. Data & Privacy	11
16. Manual Testing Checklist	12
17. Glossary	12

1 Overview & Purpose

The **Click and Align Subtitle Tool** is an Adobe Premiere Pro CEP (Common Extensibility Platform) panel extension developed by PlanetRead. It allows editors to manually time subtitles to a video by clicking through lines and marking start/end points at the playhead position, producing a fully timed SRT caption track placed directly onto the active Premiere sequence.

Unlike the Slide and Align tool (which distributes timing proportionally from a text file), this tool is designed for precise, frame-level subtitle alignment. Editors load a **.txt** or **.srt** file, step through each line using the Mark Start and Mark End buttons, and the extension handles marker placement, state persistence, and SRT export automatically.

■ This extension targets Premiere Pro 25.0+. It will not load on hosts outside the declared version range as specified in the manifest. The panel extension ID is `com.planetread.clickandalign.subtitle.panel`.

2 Architecture

The extension uses the standard CEP two-process model. The **panel layer** runs inside Chromium (HTML + JavaScript) and communicates with **ExtendScript** in Premiere's host process via `CSInterface.evalScript`. Unlike the Slide and Align tool — which uses a static `ScriptPath` in the manifest — this extension also loads its JSX at runtime via `$.evalFile` inside `loadStaticScript()`, providing an extra guarantee that definitions are present before any UI action fires.

Communication Flow

- **User** clicks a button or adjusts a control in the panel.
- **main.js** builds an ExtendScript function call string and invokes `CSInterface.evalScript()`.
- **CSInterface** serialises and relays the call to Premiere's ExtendScript engine.
- **subtitles.jsx** executes: reads/writes sequence markers, manages subtitle state, builds and imports SRT.
- A **result string** (subtitle text, filename, or status) is returned asynchronously to the panel callback.

Polling

`checkToolStatus()` polls approximately every second on `DOMContentLoaded`. It evaluates whether an active sequence exists, and combines this with panel visibility and window focus to apply `.tool-active` or `.tool-inactive` CSS on the container, giving the user a clear visual cue when the panel is ready to use.

3 Layer Reference

Layer / File	Role
<code>index.html</code>	Panel layout, controls, loads <code>CSInterface.js</code> and <code>main.js</code> .
<code>main.js</code>	<code>evalScript</code> calls, loads JSX via <code>\$.evalFile</code> once per session, manages UI state (focus, word spacing, status polling).
<code>jsx/subtitles.jsx</code>	File I/O, subtitle model (<code>subtitleLines</code> array), marker creation, SRT build, caption track import, session persistence.
<code>CSXS/manifest.xml</code>	Bundle registration: extension IDs, host version range, <code>MainPath</code> , <code>ScriptPath</code> , panel geometry, CEF flags.
<code>CSInterface.js</code>	Adobe CEP JavaScript bridge. <code>evalScript</code> is the primary cross-process call mechanism.

4 UI → ExtendScript Map

UI Element (id)	main.js Behaviour	ExtendScript Called
setText	Opens file dialog; loads script; fetches filename	\$.evalFile(.../subtitles.jsx) → main() → getCurrentFileName()
startText (Mark Start)	Fires evalScript on click	toggleStartMark() → getCurrentSubtitle()
endText (Mark End)	Fires evalScript on click	markEnd('static') → getCurrentSubtitle()
addCaptionsNow	Sets spacing if needed, fires evalScript	setWordSpacing(n) → addCaptionsNow()
reset	Fires evalScript; clears panel; resets currentFileName	resetSubtitles()
wordSpacingSlider / wordSpacing / up/down	Calls loadStaticScript then normalises value	setWordSpacing(n)

■ On DOMContentLoaded, main.js calls loadStaticScript() and applies the default word spacing via setWordSpacing, then starts the ~1 s checkToolStatus polling loop.

5

State Model

What	Where	Notes
Loaded lines, current index, per-line start/end, filename, word spacing	ExtendScript globals in subtitles.jsx: subtitleLines, currentIndex, currentFileName, wordSpacingSetting	Lives in Premiere's ExtendScript engine for the panel session lifetime.
Persisted session snapshot	Folder.userData + static_subtitle_tool_state.json	Written by saveSubtitleState after marking steps. loadSubtitleState restores it — but is not auto-called on panel open (see Known Issues §14).
Temp SRT for import	Folder.temp + temp_subtitles_.srt	Created at export time; not automatically deleted.

■ A subtitle line is exportable only when both start and end are set. autoCreateCaptionTrackFromSubtitles silently skips any line missing either timestamp.

6

Caption Export Flow

Step	Action	Branch
1	Build SRT text from all lines where start and end are set; apply applyWordSpacing using wordSpacingSetting	Always
2	Write file to Folder.temp as UTF-8, LF line endings, no BOM	Always
3	Record destBin.children.numItems before import	Always
4	Call app.project.importFiles([tempFile.fsName], true, destBin, false)	Always
5a	numItems increased → import succeeded	Happy path → createCaptionTrack
5b	numItems unchanged → first import failed	Retry path
6	Rewrite same temp path with UTF-8, CRLF endings, UTF-8 BOM	Retry path
7	importFiles again; check numItems a second time	Retry path
8	activeSequence.createCaptionTrack(importedSRT, 0)	Both paths on success
9	Alert if any step fails (empty lines, no sequence, import failure, etc.)	Error paths

7

Function Reference — main.js

Function	Purpose
safeEvalScript(script, callback)	Wraps evalScript in try/catch. On missing CEP or thrown error, calls callback(null). Used for status checks and script load.
safeGetSystemPath(pathType)	Wraps getSystemPath; returns null if CEP unavailable. Used by loadStaticScript to locate jsx/subtitles.jsx.
loadStaticScript()	Ensures ExtendScript definitions from subtitles.jsx are loaded once per panel lifetime via \$.evalFile. Sets staticScriptLoaded = true after attempt, even if extension path is missing.
normalizeWordSpacing(value)	parseFloat; if NaN or < 1 → 1; if > 15 → 15. Returns clamped float.
updateWordSpacing(spacing)	Syncs slider and number input (toFixed(1)); calls loadStaticScript then evalScript(setWordSpacing(normalized)).
updateTextBox(subtitleText)	Sets #subtitleText textarea value directly. Note: called with 0 after main() — see Known Issues §14.
checkToolStatus()	Evaluates whether active sequence exists; combines with hasWindowFocus and document.visibilityState to determine active/inactive. Calls updateToolStatus.
updateToolStatus(active)	Toggles .tool-active / .tool-inactive on .container when derived state changes (reduces DOM churn).

8

Function Reference — subtitles.jsx (Utilities & I/O)

Utilities and File I/O

Function	Purpose
logEvent(msg)	\$.writeln + optional app.setSDKEventMessage for in-app diagnostics and host console logging.
readFileAsBinary(file)	Reads file as BINARY mode, returning a byte string (one char per byte, values 0–255).
decodeUTF8(byteString)	Full UTF-8 decoder: handles BOM, multi-byte sequences, and surrogate pairs. Invalid bytes replaced with U+FFFD.
readTextFileTolerant(file)	Sniffs BOM; tries UTF-8 then UTF-16 text reads until content is successfully retrieved.
normalizeAndFilterLines(rawText)	Normalises newlines, trims and splits into non-empty lines. Used for .txt file mode.

Spacing, Modes & Parsing

Function	Purpose
applyWordSpacing(line, spacing)	Replaces space runs with floor(spacing) normal spaces plus thin spaces (U+2009) for the fractional part (min 1 thin space if fractional > 0).
setWordSpacing(value)	Clamps and stores wordSpacingSetting for use at export time.
setSubtitleMode(mode)	Accepts 'static' only; shows alert. Legacy hook — not actively used.
parseSRT(srtText)	Parses SubRip blocks into { text, start, end } time objects for .srt file mode.
main()	File dialog for .txt / .srt; fills subtitleLines array; .txt lines get start/end null; .srt entries get parsed times. Sets currentIndex = 0 and calls updateTextBoxWithCurrentSubtitle.

9

Function Reference — subtitles.jsx (Marking & Export)

Marking and Markers

Function	Purpose
toggleStartMark()	Two-click Mark Start state machine. First click sets start + point marker. Second click sets end, replaces marker span, advances index, optionally starts next line. See §10 for full state table.
markEnd(mode)	Requires start set. Sets end, inserts full marker span, advances index. Does NOT auto-start next line at playhead. See §11.
insertSubtitleMarker(startTime, endTime, text)	seq.markers.createMarker at startTime.seconds; name = 'Static Subtitle'; comments = text; marker.end = endTime.seconds. Start-only marks use startTime === endTime (point marker).
markStart(mode)	Legacy function mirroring toggleStartMark logic. Not wired to any UI button — safe to remove.

UI Sync and Getters

Function	Purpose
getCurrentSubtitle()	Returns text for currentIndex, or empty string if past end.
getCurrentFileName()	Returns currentFileName or empty string.
updateTextBoxWithCurrentSubtitle()	Pushes current line to app.setTextBox if defined; used after marking advances the index.

Time Formatting and Export

Function	Purpose
pad(num, size)	Zero-pads integers for SRT timestamp strings.
formatTime(t)	Converts a Premiere Time object to HH:MM:SS,mmm string for SRT output.
addCaptionsNow()	Thin wrapper: logs via logEvent then calls autoCreateCaptionTrackFromSubtitles(). Use when exporting before all lines are marked.
autoCreateCaptionTrackFromSubtitles()	Full export pipeline: build SRT from completed lines → write temp file → import with retry → createCaptionTrack. See §6 for step-by-step flow.

Reset and Persistence

Function	Purpose
resetSubtitles()	Clears subtitleLines, currentIndex, currentFileName, wordSpacingSetting. Removes all markers named 'Static Subtitle' from the active sequence. Calls clearSubtitleState().
saveSubtitleState()	Writes { subtitleLines, currentIndex, mode: 'static' } as JSON to Folder.userData/static_subtitle_tool_state.json (UTF-8). Called after marking steps.
loadSubtitleState()	Reads and restores from JSON if file exists. Currently not called automatically on panel open — see Known Issues §14.
clearSubtitleState()	Deletes the JSON persistence file if it exists.

10 Mark Start State Machine

`toggleStartMark()` is the core of the two-click Mark Start workflow. It reads the current playhead position from `app.project.activeSequence.getPlayerPosition()` and branches on the current line's state:

Current Line State	Action Taken
start === null	Set start = current playhead time. Call <code>insertSubtitleMarker(time, time, text)</code> to place a point marker. Refresh UI with <code>updateTextBoxWithCurrentSubtitle</code> .
start set, end === null	Validate <code>time.seconds > start.seconds</code> . Set end = time. Replace point marker with full span via <code>insertSubtitleMarker(start, time, text)</code> . Increment <code>currentIndex</code> . If more lines remain: set next line's start = time and insert a point marker for it. Call <code>saveSubtitleState</code> . If this was the last line: call <code>autoCreateCaptionTrackFromSubtitles</code> and return.
start and end both set	No-op. This state should not occur for the active index in normal use; the toggle has no branch for it.

■ The key behaviour of the second click is that it automatically chains the next subtitle: the next line's start is pre-set to the same time as the current line's end, so the editor only needs to click Mark End for the next line (no separate Mark Start click required), unless they want to reposition the start point.

11 Mark End Behaviour

`markEnd('static')` is the simpler of the two marking functions. Key distinctions from the second click of `toggleStartMark`:

Aspect	markEnd behaviour
Precondition	Active sequence required. <code>currentIndex</code> must be within <code>subtitleLines</code> . Current line must have start !== null — otherwise alerts 'Mark Start first'.
End time validation	<code>time.seconds</code> must be strictly greater than <code>start.seconds</code> .
Marker	Calls <code>insertSubtitleMarker(start, end, text)</code> to create or update the full-span marker.
Index advance	Increments <code>currentIndex</code> after setting end.
Next line chaining	Does NOT set the next line's start to the current playhead. The next line remains unstarted until the user clicks Mark Start again.
Completion	If <code>currentIndex >= subtitleLines.length</code> after advance: calls <code>autoCreateCaptionTrackFromSubtitles</code> . Otherwise: <code>updateTextBoxWithCurrentSubtitle</code> and <code>saveSubtitleState</code> .

12 Extension Manifest

The manifest at CSXS/manifest.xml is the authoritative registration source. Changes here affect loading, debugging eligibility, and Premiere's host compatibility check.

Property	Value
Bundle ID	com.planetread.clickandalign.subtitle
Bundle Version	1.0.0
Panel Extension ID	com.planetread.clickandalign.subtitle.panel
Premiere Host	PPRO 25.0+
CEP Runtime	CSXS 12.0
MainPath	./index.html
ScriptPath	./jsx/subtitles.jsx
Default Panel Size	400 × 800 px
CEF Flags	--enable-nodejs --mixed-context
Menu Label	Window → Extensions → Click and Align Subtitle Tool

■ ScriptPath causes Premiere to load subtitles.jsx at panel startup. The panel also calls \$.evalFile for the same file inside loadStaticScript() — this is intentional: it ensures definitions are present even if the ScriptPath load occurred before the panel was ready to use them.

■ --enable-nodejs grants Node.js APIs (fs, path, child_process) in the Chromium process. --mixed-context allows Node.js and browser globals to coexist. Both increase capability and attack surface — avoid loading untrusted remote scripts.

13 Debugging Guide

13.1 Panel (main.js) — Chromium DevTools

- Enable **PlayerDebugMode** for the matching CSXS major version in the OS registry (Windows) or plist (macOS).
- Place a **.debug** file in the extension root alongside CSXS/. List the extension ID com.planetread.clickandalign.subtitle.panel and host PPRO.
- Restart Premiere Pro and open the panel via Window → Extensions.
- Navigate to `http://127.0.0.1:<port>` in Chrome or Edge to attach DevTools.
- Use `console.log` in main.js freely — output appears in the DevTools Console tab.

13.2 ExtendScript (subtitles.jsx)

Method	Usage
\$.writeln(msg)	Writes to the ExtendScript Toolkit console or host debug log when attached.
logEvent(msg)	Calls \$.writeln and app.setSDKEventMessage('info') for lightweight in-Premiere diagnostics.
Return string inspection	All major functions return strings. Log them in main.js via the safeEvalScript callback or evalScript callback.
app.setSDKEventMessage(msg, 'info')	Visible in Premiere's Events panel depending on build — useful for tracing autoCreateCaptionTrackFromSubtitles steps.

13.3 Session State Debugging

The persistence file at `Folder.userData/static_subtitle_tool_state.json` can be inspected directly in a text editor to verify what state was saved. If the panel loads with unexpected subtitle data, delete this file and reload — it is equivalent to calling `clearSubtitleState()`.

14 Known Issues & Follow-ups

Issue	Detail	Recommended Action
Session restore not wired	loadSubtitleState() is implemented in subtitles.jsx but not called from main.js. State is saved after marking, but reopening the panel does not auto-restore JSON.	Wire loadSubtitleState() call after loadStaticScript() on DOMContentLoaded.
updateTextBox called with 0	After main(), main.js calls updateTextBox(0) — setting the textarea to the string '0', not the first subtitle line. The code comment is misleading.	Replace with evalScript('getCurrentSubtitle()', cb) + updateTextBox(text) to show the first line immediately after file pick.
Dead code: markStart(mode)	markStart is in subtitles.jsx but not wired to any UI button. Logic mirrors toggleStartMark.	Remove, or wire intentionally with a documented mode parameter.
README vs manifest bundle ID	Any user-facing docs should reference com.planetread.clickandalign.subtitle, not older or mismatched IDs.	Audit README and any install guides; align to the manifest value.

15 Data & Privacy

Data Type	Location	Notes
Temp SRT file	Folder.temp — temp_subtitles_.srt	Created per export. Not automatically deleted; clean periodically if disk space is a concern.
Session JSON	Folder.userData — static_subtitle_tool_state.json	Written after each marking step. Cleared on Reset. Not encrypted; avoid on shared machines with sensitive scripts.
Input .txt / .srt	User-selected local path; read once during main()	Not copied or transmitted. Path passed as a string through the evalScript call only.
Network activity	None by the subtitle workflow	CSInterface.js (Adobe library) may contain generic XHR helpers; this extension does not invoke them for subtitle features.

16

Manual Testing Checklist

■	Install / load	Extension appears under Window → Extensions; panel opens without JS errors.
■	TXT — UTF-8	UTF-8 file loads; empty lines skipped; line count alert shown; first line ready for marking.
■	TXT — UTF-16	UTF-16 encoded file loads correctly via readTextFileTolerant.
■	SRT load	SRT file loads; marking updates timing from playhead; existing cue times are preserved unless overwritten.
■	Mark Start — first click	Sets start, places point marker on timeline named 'Static Subtitle'.
■	Mark Start — second click	Sets end, advances to next line, pre-sets next line's start, adds point marker for next line.
■	Mark Start + Mark End	Mark End does not auto-start next line at playhead; next line waits for a fresh Mark Start click.
■	Word spacing	Slider/input updates export spacing; captions reflect thin-space padding in created track.
■	Add Completed Captions	Only lines with both start and end are exported; caption track created on sequence.
■	Import retry	CRLF + BOM retry path fires when first import attempt fails (bin count unchanged).
■	Reset	Clears state, removes all 'Static Subtitle' markers, JSON file deleted, panel cleared.
■	Tool status styling	With sequence open and panel focused/visible: .tool-active applied. When blurred or hidden: .tool-inactive applied.
■	No active sequence	Mark Start / Mark End surface appropriate alerts; no crash; no orphaned markers.
■	Persistence (manual)	After marking several lines, close and reopen the panel; call loadSubtitleState manually via evalScript to verify JSON restore works correctly.

17 Glossary

Term	Definition
CEP	Common Extensibility Platform — Adobe's framework for building HTML/JS panels that run inside Creative Cloud host applications.
ExtendScript	Adobe's ECMAScript 3-based scripting engine (.jsx files) running inside the host application process with direct access to the Premiere Pro DOM.
CSInterface	JavaScript bridge library providing evalScript and other helpers for panel-to-host communication.
evalScript	CSInterface method that serialises an ExtendScript string, sends it to the host process, evaluates it, and returns the result asynchronously via callback.
\$.evalFile	ExtendScript global that loads and evaluates a .jsx file into the host engine, defining all its global functions. Called once per session by loadStaticScript().
SRT	SubRip Subtitle format (.srt) — numbered caption blocks with start/end timestamps and text lines.
subtitleLines	The core ExtendScript array. Each entry: { text, start, end } where start/end are Premiere Time objects (or null for unstarted .txt lines).
toggleStartMark	The two-click Mark Start function. First click = set start; second click = set end and advance. See §10.
markEnd	The Mark End function. Sets end only; does not auto-chain the next subtitle start. See §11.
Point marker	A sequence marker where start === end (zero duration). Used as a placeholder when only the start time has been set, replaced with a full span on the second click.
Thin space (U+2009)	Unicode narrow space used by applyWordSpacing to add fractional inter-word padding in caption text.
CRLF	Carriage Return + Line Feed (\r\n) — Windows line-ending convention required by some Premiere SRT import paths (used in retry).
BOM	Byte Order Mark — UTF-8 BOM (EF BB BF) prepended to the retry SRT file to declare encoding.
Folder.userData	ExtendScript global resolving to the OS user data directory. Used for the session JSON persistence file.
Folder.temp	ExtendScript global resolving to the OS temp directory. Used for the ephemeral SRT export file.
CSXS	Creative Suite Extension Services — Adobe infrastructure for CEP extensions, referenced in manifest.xml and .debug files.